

Extracting Structured Understanding from Large Language Models

Abstract

Large language models (LLMs) are trained on massive corpora of human-generated text, including scientific and technical literature. Although the training objective is purely statistical—next-token prediction—the resulting models exhibit non-trivial competence in many scientific domains. This suggests that the weights and internal activations of such models implicitly encode aspects of the underlying structure of the world that constrains scientific discourse.

This document formulates, in a precise way, the hypothesis that LLMs contain latent representations corresponding to real-world mechanisms and structures, motivated by structural realism and representation learning. It proposes an operational notion of “understanding” in this context, outlines a research architecture for identifying and extracting such structure, and provides pseudocode for a generic Python system that interrogates LLMs and builds explicit, externalized models (graphs, axioms, causal structures) which can be further formalized and analyzed.

1 Language Models as Compressed Predictive Codes

Let $\mathcal{T}$ denote the space of token sequences in some language (or set of languages), and let $\mathcal{D}$ denote a large corpus of text, including scientific and technical material. A transformer-style LLM with parameters θ is trained to approximate the conditional distribution

$$p_{\theta}(t_k \mid t_1, \dots, t_{k-1}) \approx p_{\text{data}}(t_k \mid t_1, \dots, t_{k-1}), \quad (1)$$

by minimizing a cross-entropy or negative log-likelihood objective over $\mathcal{D}$.

Conceptually, p_{θ} is a compressed code of the empirical distribution of token sequences encountered during training. Because scientific and technological texts are constrained by regularities of the physical and social world (experiments, engineering constraints, interventions, etc.), the learned distribution must implicitly reflect those regularities whenever they are necessary for successful prediction.

The internal computation of a transformer defines, at each layer ℓ and position k , a hidden state

$$h_k^{(\ell)} \in \mathbb{R}^d, \quad (2)$$

which can be viewed as a point in a high-dimensional *latent space*. Self-attention and feed-forward sublayers map previous hidden states to new ones, ultimately producing logits over the vocabulary.

From the perspective of representation learning, the family of hidden states $\{h_k^{(\ell)}\}$ is an internal representation of the conditional structure of text. To the extent that text is shaped by real-world mechanisms, these representations are candidates for encoding aspects of those mechanisms in a distributed form.

2 Scientific Discourse and Real-World Structure

Scientific discourse is not an arbitrary collection of symbol sequences. It is tightly constrained by:

- empirical regularities and experimental outcomes,
- engineering and technological constraints,
- mathematical structures used to model phenomena,
- communal norms of explanation, prediction, and control.

Concepts and terms in scientific texts include:

- directly referential entities (e.g. “electron”, “DNA”, “neuron”),
- operationally defined quantities (e.g. temperature as defined by a given instrument),
- idealizations and effective constructs (e.g. ideal gas, infinite lattice),
- historical or metaphorical remnants.

The mapping from linguistic items to the world is therefore indirect and context-dependent.

What tends to be more stable across theoretical and linguistic changes is *structure*:

- symmetries and invariants,
- causal dependencies,
- functional roles within systems,
- mathematical relationships (differential equations, conservation laws, constraints).

A structural-realist view suggests that what is most reliably “pointed to” by long-run scientific practice is this relational structure rather than the intrinsic nature of individual entities. If an LLM is trained on a large enough portion of such practice, its internal representations are likely to encode approximations to these structures, insofar as they are necessary for accurate prediction of scientific text.

3 Operational Notions of Understanding

To make the notion of “understanding” in an LLM precise and empirically tractable, we adopt an operational stance. At least three complementary notions are relevant.

3.1 Structural Understanding (Internal Mechanisms)

A model displays structural understanding of some domain D if there exist:

- identifiable latent variables (features, subspaces, attention patterns) whose behavior corresponds to well-defined concepts or quantities in D ,
- identifiable circuits (subgraphs of the computation) that implement recognizable algorithms or reasoning steps in D ,

and these identifications can be validated by causal interventions:

- ablating or modifying these variables changes behavior in predictable ways,
- activation patching or counterfactual interventions produce controlled changes in outputs.

3.2 Behavioral Understanding (Task-Performance View)

A model displays behavioral understanding of D if, when probed exclusively through its input-output behavior, it:

- answers questions and solves problems in D with high accuracy,
- generalizes robustly to novel but structurally similar tasks,
- maintains internal consistency under rephrasings and counterfactuals,

- exhibits coherent explanatory and predictive behavior aligned with the underlying structure of D .

This does not require any insight into internal representations; it is entirely defined by observable performance.

3.3 Extractable Knowledge (Explicit Representations)

A model’s understanding of D is *extractable* if one can algorithmically derive from it an explicit representation of a theory or world-model for D , for example:

- a knowledge graph of entities and relations,
- a set of axioms and theorems in a formal system,
- a causal graph or structural causal model,
- a simulation model (differential equations, update rules).

This representation should be:

- significantly more compact than the full LLM,
- sufficient to reproduce key capabilities on tasks within D ,
- interpretable to human experts or analyzable by other systems.

4 Latent-Space Structural Realism Hypothesis

We now formalize a central working hypothesis.

Hypothesis (Latent-Space Structural Realism; LSSR). *Let D be a scientific domain whose practice is well-represented in the training corpus of a large language model p_θ . Suppose that D is governed by relatively stable underlying mechanisms and structures (e.g. laws of motion, genetic inheritance, chemical reaction networks). Then there exist latent features, subspaces, and circuits within p_θ whose causal functional roles are isomorphic, in an appropriate sense, to aspects of these mechanisms and structures.*

This hypothesis is not about a one-to-one mapping between individual neurons and real-world objects, but about the existence of internal representations whose *geometry* and *causal role* correspond to structural properties of the world required to predict scientific text.

For LSSR to have empirical content, at least two conditions must be satisfied.

4.1 Cross-Corpus Invariance

Consider multiple LLMs trained on different but overlapping corpora that describe the same domain D :

- different natural languages,
- different mixtures of textbooks, research articles, and code,
- different institutional and stylistic conventions.

If all these models exhibit internal features or subspaces that can be aligned (up to invertible transformations) and that participate in analogous computations when solving problems in D , then these internal structures are strong candidates for representations of domain-level regularities rather than artifacts of any particular corpus.

4.2 Causal Interpretability and Intervention

For a putative internal representation z to count as encoding some mechanism M in D , the following should hold:

- *Decodability*: z is predictive of relevant quantities associated with M across many contexts (linear or simple nonlinear probes suffice).
- *Causal relevance*: modifying z (e.g. via activation editing) causes systematic and interpretable changes in the model’s outputs regarding M while leaving unrelated behavior largely intact.
- *Stability*: the relationship between z and M persists across a range of inputs, prompts, and tasks within D .

Under these conditions, it is meaningful to say that the model’s latent space contains a representation of M in the sense relevant for LSSR.

5 Research Architecture for Extracting Understanding

To make LSSR testable and to extract explicit models of domains from LLMs, we outline a modular research architecture.

5.1 Step 1: Domain Selection and Ground-Truth Theories

Choose a domain D with:

- a reasonably well-understood formal theory (e.g. Newtonian mechanics, Mendelian genetics, basic chemical kinetics),
- a significant footprint in the training corpus (textbooks, problem sets, research articles).

Let $\mathcal{M}_D$ denote an explicit, externalized model of the domain: a set of equations, axioms, causal graphs, or other formal structures specified independently of any LLM.

5.2 Step 2: Behavioral Extraction via Interrogation

Treat the LLM as an oracle and interrogate it to extract an explicit approximation $\hat{\mathcal{M}}_D$ of $\mathcal{M}_D$.

Design prompt families that elicit:

- *Concept inventories*: lists of key entities, quantities, and concepts in D .
- *Prerequisite relations*: which concepts depend on which others.
- *Causal structures*: variables and their causal dependencies, including interventions and counterfactuals.
- *Axioms and laws*: candidate minimal sets of principles from which others can be derived.

Aggregate multiple runs with paraphrased prompts and sampling schemes, then post-process the raw textual responses into:

- a concept graph (nodes as concepts, edges as prerequisite or definitional relations),
- a causal graph or set of structural equations,
- a corpus of axioms, laws, and example derivations.

This yields a structured object $\hat{\mathcal{M}}_D$ that can be stored (e.g. as JSON, a graph database, or input to a proof assistant).

5.3 Step 3: Mechanistic Extraction via Probing and Circuits

For an open-source LLM (or local model) with accessible internal states, collect activations $h_k^{(\ell)}$ on carefully constructed prompts in D , and:

- train linear or shallow nonlinear probes to decode relevant quantities (e.g. energies, momenta, genotype frequencies) from $h_k^{(\ell)}$,
- apply feature-extraction methods (e.g. sparse autoencoders) to discover latent directions associated with interpretable concepts,

- identify attention heads and MLP components that are necessary for correct reasoning in D via ablation and activation patching.

This yields a family of internal features and circuits that appear to implement computations corresponding to aspects of $\mathcal{M}_D$ or $\hat{\mathcal{M}}_D$.

5.4 Step 4: Alignment Between External and Internal Models

Establish correspondences between elements of the explicit model $\hat{\mathcal{M}}_D$ and internal structures:

- map concepts in $\hat{\mathcal{M}}_D$ to probe directions and feature vectors,
- map laws or update rules to observed computation patterns across layers and heads,
- test whether perturbing the internal representation of a concept or law yields the expected change in behavior within the explicit model.

The goal is to obtain a functorial correspondence between:

$$(\text{Category of internal circuits and features}) \longrightarrow (\text{Category of explicit domain models}).$$

5.5 Step 5: Information-Theoretic Formulation

From an information-theoretic perspective, let C_D denote the (hypothetical) Kolmogorov complexity of an optimal world-model for domain D . The full LLM has a parameter description length $|\theta|$ that is vastly larger than C_D , because it models many domains and linguistic phenomena simultaneously.

Define $K_{\text{LLM}}^*(D)$ as the minimal description length of a distilled model plus external representation that:

- reproduces the LLM’s performance on tasks restricted to D up to a specified tolerance,
- is expressed in a chosen formal language (e.g. a proof assistant, a probabilistic programming language, or a DSL for dynamics).

Estimating $K_{\text{LLM}}^*(D)$ by compression and distillation experiments provides a quantitative measure of how much structured information about D is encoded and how efficiently it can be externalized.

6 System Design and Pseudocode

We now describe a generic system architecture, implemented in Python, that:

- interacts with an LLM via a generic interface (CLI tool or API),
- performs behavioral interrogation to build a structured external model $\hat{\mathcal{M}}_D$,
- optionally orchestrates mechanistic experiments for models with accessible internals.

6.1 System Components

- C1 LLM Interface:** abstract class encapsulating how prompts are sent and responses received (CLI wrapper or HTTP API client).
- C2 Domain Configuration:** specification of domain D , including prompt templates, parsing rules, and evaluation benchmarks.
- C3 Interrogation Engine:** constructs prompts, queries the LLM, and collects raw textual responses.
- C4 Structure Extractor:** parses responses into structured representations (graphs, axioms, causal models).
- C5 Formalization Backend:** optional component that translates the structured representation into a formal language (e.g. Lean, Coq, a DSL).

C6 Evaluation Module: evaluates the extracted model $\hat{\mathcal{M}}_D$ against ground-truth problems and criteria.

C7 Mechanistic Module (optional): hooks into local models to run probing and circuit analysis.

6.2 Pseudocode

The following pseudocode is written in Python-like style but abstracts away implementation details (error handling, logging, concurrency, etc.). It is designed so that the `LLMInterface` can later be backed either by a CLI binary or by an API client.

```
# -----
# Core abstractions
# -----

from dataclasses import dataclass
from typing import List, Dict, Any, Optional

class LLMInterface:
    """
    Abstract interface to a language model.

    Implementations may call:
    - a local CLI binary (e.g. via subprocess),
    - a remote HTTP API,
    - a local Python model in process.
    """
    def generate(self, prompt: str, **kwargs) -> str:
        raise NotImplementedError

    @dataclass
    class DomainConfig:
        name: str
        description: str
        # Prompt templates for interrogation
        concept_prompt_template: str
        prereq_prompt_template: str
        causal_prompt_template: str
        axiom_prompt_template: str
        # Evaluation problems and ground-truth solutions
        benchmarks: List[Dict[str, Any]]

    @dataclass
    class StructuredModel:
        """
        Externalized approximation to  $\mathcal{M}_D$  extracted from the LLM.
```

```

"""
concepts: List[Dict[str, Any]]
concept_graph: Dict[str, List[str]] # adjacency list: concept -> prerequisites
causal_graph: Dict[str, List[str]] # variable -> parents
axioms: List[str]
metadata: Dict[str, Any]

# -----
# Interrogation Engine
# -----

class InterrogationEngine:
def __init__(self, llm: LLMInterface, domain: DomainConfig):
self.llm = llm
self.domain = domain

def _ask(self, template: str, **fields) -> str:
prompt = template.format(**fields)
return self.llm.generate(prompt)

def collect_concepts(self) -> str:
return self._ask(self.domain.concept_prompt_template,
domain_description=self.domain.description)

def collect_prerequisites(self, concept_list_str: str) -> str:
return self._ask(self.domain.prereq_prompt_template,
domain_description=self.domain.description,
concept_list=concept_list_str)

def collect_causal_structure(self, concept_list_str: str) -> str:
return self._ask(self.domain.causal_prompt_template,
domain_description=self.domain.description,
concept_list=concept_list_str)

def collect_axioms(self, concept_list_str: str) -> str:
return self._ask(self.domain.axiom_prompt_template,
domain_description=self.domain.description,
concept_list=concept_list_str)

# -----
# Structure Extraction
# -----

class StructureExtractor:
"""
Converts raw textual responses into structured objects.

```

In practice, this will use parsing, pattern matching, and possibly additional LLM calls for strict formatting.

```
"""
```

```
def parse_concepts(self, raw_concepts: str) -> List[Dict[str, Any]]:
# Example: parse bullet lists "1. Name: description"
# Return list of { "name": ..., "description": ... }
raise NotImplementedError
```

```
def build_concept_graph(self,
concepts: List[Dict[str, Any]],
raw_prereqs: str) -> Dict[str, List[str]]:
# Parse prerequisite relations from raw_prereqs and
# align them with the concept names.
raise NotImplementedError
```

```
def build_causal_graph(self,
concepts: List[Dict[str, Any]],
raw_causal: str) -> Dict[str, List[str]]:
# Parse causal relations and align with concepts.
raise NotImplementedError
```

```
def parse_axioms(self, raw_axioms: str) -> List[str]:
# Extract candidate axioms/laws from text.
raise NotImplementedError
```

```
def assemble_model(self,
domain: DomainConfig,
raw_concepts: str,
raw_prereqs: str,
raw_causal: str,
raw_axioms: str) -> StructuredModel:
concepts = self.parse_concepts(raw_concepts)
concept_graph = self.build_concept_graph(concepts, raw_prereqs)
causal_graph = self.build_causal_graph(concepts, raw_causal)
axioms = self.parse_axioms(raw_axioms)
metadata = {
    "domain_name": domain.name,
    "description": domain.description
}
return StructuredModel(
    concepts=concepts,
    concept_graph=concept_graph,
    causal_graph=causal_graph,
    axioms=axioms,
    metadata=metadata,
)
```



```

# -----
# Evaluation Module
# -----

class Evaluator:
    """
    Evaluates a StructuredModel using benchmark problems.
    Benchmarks may be symbolic, numerical, or qualitative.
    """

    def __init__(self, domain: DomainConfig):
        self.domain = domain

    def evaluate(self, model: StructuredModel) -> Dict[str, Any]:
        results = []
        for problem in self.domain.benchmarks:
            # Example structure of a benchmark:
            # { "type": "prediction", "input": ..., "expected": ... }
            result = self._evaluate_single(problem, model)
            results.append(result)
        summary = self._aggregate(results)
        return {"per_problem": results, "summary": summary}

    def _evaluate_single(self,
        problem: Dict[str, Any],
        model: StructuredModel) -> Dict[str, Any]:
        # Use model.concepts, model.causal_graph, model.axioms to
        # compute or infer an answer, then compare to expected.
        raise NotImplementedError

    def _aggregate(self, results: List[Dict[str, Any]]) -> Dict[str, Any]:
        # Compute summary statistics, e.g. accuracy, coverage.
        raise NotImplementedError

# -----
# Optional: Mechanistic Module
# -----

class MechanisticModule:
    """
    For local models where internal activations are accessible,
    orchestrate probing and circuit analysis experiments.
    """

    def __init__(self, local_model: Any):
        self.model = local_model

```

```

def collect_activations(self, prompts: List[str]) -> Any:
# Run model on prompts and record activations per layer.
raise NotImplementedError

def train_probes(self, activations: Any,
labels: Any) -> Any:
# Train linear/shallow classifiers to decode domain quantities.
raise NotImplementedError

def identify_circuits(self, activations: Any,
importance_metric: str = "ablation") -> Any:
# Identify attention heads/MLP units critical for domain tasks.
raise NotImplementedError

# -----
# High-level orchestration
# -----

def extract_structured_model(llm: LLMInterface,
domain: DomainConfig) -> StructuredModel:
interrogator = InterrogationEngine(llm, domain)
extractor = StructureExtractor()

raw_concepts = interrogator.collect_concepts()
raw_prereqs = interrogator.collect_prerequisites(raw_concepts)
raw_causal = interrogator.collect_causal_structure(raw_concepts)
raw_axioms = interrogator.collect_axioms(raw_concepts)

model = extractor.assemble_model(domain,
raw_concepts,
raw_prereqs,
raw_causal,
raw_axioms)
return model

def main():
# 1. Instantiate an LLM interface, e.g. CLI-based or API-based.
llm = SomeConcreteLLMInterface(...)

# 2. Define the domain configuration.
domain = DomainConfig(
name="Classical Mechanics (Point Particles)",
description="Newtonian mechanics of point masses in  $\mathbb{R}^3$  "
"under smooth forces.",
concept_prompt_template=(

```

```

"You are an expert in {domain_description}.\n"
"List the fundamental concepts and quantities in this domain.\n"
"For each, give a brief definition."
),
prereq_prompt_template=(
"Given the following list of concepts in {domain_description}:\n"
"{concept_list}\n\n"
"For each concept, list which other concepts are prerequisites."
),
causal_prompt_template=(
"Given the following concepts in {domain_description}:\n"
"{concept_list}\n\n"
"Describe the key causal relationships between the associated "
"variables. Output a list of 'cause -> effect' relations."
),
axiom_prompt_template=(
"Given the concepts:\n"
"{concept_list}\n\n"
"Propose a minimal set of axioms or laws for "
"{domain_description} from which the others can be derived."
),
benchmarks=[
# Populate with domain-specific problems.
],
)

# 3. Extract a structured model from the LLM.
structured_model = extract_structured_model(llm, domain)

# 4. Evaluate the extracted model.
evaluator = Evaluator(domain)
evaluation_results = evaluator.evaluate(structured_model)

# 5. (Optional) Save results, export to a formalization backend, etc.
save_structured_model(structured_model, path="mechanics_model.json")
save_evaluation(evaluation_results, path="mechanics_eval.json")

if __name__ == "__main__":
    main()

```

The pseudocode above separates the concerns of:

- interacting with an LLM (abstracted by `LLMInterface`),
- interrogating the model using domain-specific prompts,
- parsing and assembling structured external models,
- evaluating these models against explicit benchmarks,
- optionally, performing mechanistic experiments on local models.

Concrete implementations of the abstract methods can be developed incrementally, starting with simple text parsing and CLI-based interaction, and later upgraded to robust parsers, API-based interfaces, and mechanistic tooling.